

4.1 Thuật toán CART là gì?

- Scikit-Learn sử dụng thuật toán **CART** (Classification and Regression Tree) để huấn luyện cây.
- Thuật toán này khá đơn giản:
 - ➊ Phân chia tập huấn luyện thành 2 tập con sử dụng 1 đặc trưng k và ngưỡng t_k .
 - ➋ Chọn k và t_k sao cho tạo ra 2 tập con thuần khiết nhất (tức là Gini Impurity nhỏ nhất có thể).
 - ➌ Lặp lại đệ quy với từng tập con.
 - ➍ Dừng lại khi đạt đến `max_depth` hoặc không thể giảm thêm độ vẩn đục.

4.2 Hàm Chi phí của CART (Phân loại)

- Thuật toán CART sẽ tối thiểu hóa hàm chi phí (Cost Function) tại mỗi bước chia:

$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

- Trong đó:
 - G_{left}, G_{right} là độ vẩn đục Gini của tập con bên trái/phải.
 - m_{left}, m_{right} là số lượng mẫu của tập con.
 - m là tổng số lượng mẫu tại nút đang xét.
- CART là thuật toán **Tham lam (Greedy Algorithm)**. Nó chỉ tìm kiếm mức phân tách tốt nhất tại *hiện tại*, không nhìn về tương lai. Do đó không bảo đảm tối ưu cục bộ.

4.3 Gini Impurity hay Information Entropy?

- Mặc định, CART dùng **Gini Impurity** (criterion="gini").
- Tuy nhiên, ta có thể đổi sang dùng **Entropy** (criterion="entropy").
- Entropy bằng 0 khi tập hợp hoàn toàn thuần khiết (chỉ chứa 1 lớp).
- Công thức Shannon's Entropy:

$$H_i = - \sum_{k=1, p_{i,k} \neq 0}^n p_{i,k} \log_2(p_{i,k})$$

4.4 So sánh Gini và Entropy

- Vậy nên sử dụng cái nào?
- Hầu hết mọi trường hợp, **Gini và Entropy tạo ra những cây rất giống nhau.**
- Gini tính toán **nhANH hơn một chút** (vì không chứa phép Logarit), là lựa chọn mặc định tốt.
- Entropy đôi khi có xu hướng tạo ra các cây cân bằng hơn (balanced trees) trong khi Gini thường cắt ra nhánh lớn nhất.

4.5 Độ phức tạp Thời gian (Time Complexity)

- **Quá trình Suy luận (Dự đoán):**

- Phải duyệt từ gốc đến lá. Độ sâu là $O(\log_2(m))$.
- Phức tạp dự đoán cực nhanh: $O(\log_2(m))$ độc lập với số lượng đặc trưng!

- **Quá trình Huấn luyện:**

- Thuật toán so sánh mọi đặc trưng (hoặc `max_features` nếu được cài) trên mọi ngưỡng.
- Phức tạp thời gian là $O(n \times m \log_2(m))$.
- Rất chậm đối với các tập dữ liệu có m lớn (Hàng triệu dòng).

5.1 Tại sao phải Chính quy hóa?

- Cây quyết định là mô hình **Non-parametric** (Phi tham số).
- Nghĩa là số lượng tham số không được ấn định trước, dẫn đến việc mô hình có thể phát triển tự do theo dữ liệu.
- Nếu không có giới hạn, cây sẽ cổ phân nhánh đến khi mỗi chiếc lá chỉ có 1 điểm duy nhất (Tương đương việc ghi nhớ học vẹt bộ dữ liệu).
- Điều này gây ra hiện tượng **Overfitting** cực kỳ nghiêm trọng.

5.2 Các Siêu tham số Chính quy hóa

- `max_depth`: Độ sâu tối đa của cây (Default: None). Giảm nó để hạn chế Overfitting.
- `min_samples_split`: Số lượng mẫu tối thiểu một nút phải có trước khi được phân chia.
- `min_samples_leaf`: Số mẫu tối thiểu một nút lá phải có.
- `max_leaf_nodes`: Số lượng nút lá tối đa.
- `max_features`: Số lượng đặc trưng tối đa được xem xét tại mỗi lần chia.

5.3 Hiệu ứng của Chính quy hóa

- Hình bên trái: Cây không giới hạn ($\text{min_samples_leaf}=1$). Ranh giới ngoằn ngoèo, Overfitting.
- Hình bên phải: Cây bị ràng buộc ($\text{min_samples_leaf}=4$). Ranh giới mượt mà hơn, khả năng tổng quát hóa (Generalization) tốt hơn trên dữ liệu mới.

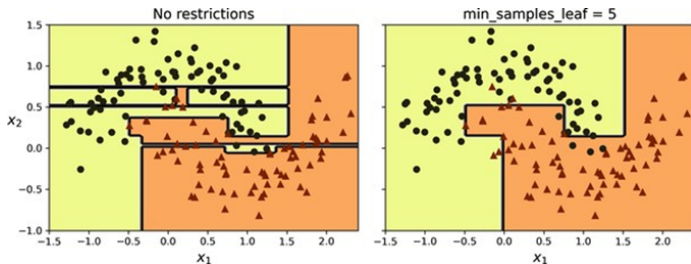


Figure: Hình 6-3: Cây không chính quy (trái) vs Chính quy hóa (phải)

6.1 Lớp DecisionTreeRegressor

- Cây quyết định cũng giải quyết xuất sắc các bài toán Hồi quy.
- Lúc này, thay vì dự đoán một "nhãn lớp", nút lá sẽ dự đoán một "giá trị liên tục" (Trung bình cộng của tất cả các mẫu nằm trong lá đó).
- Sử dụng hàm mục tiêu là MSE (Mean Squared Error) để tối ưu.

6.2 Code: Hồi quy dữ liệu Quadratic

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

# Tao du lieu mo phong y = x^2 + nhieu
np.random.seed(42)
X_quad = np.random.rand(200, 1) - 0.5
y_quad = X_quad ** 2 + 0.025 * np.random.randn(200, 1)

# Huan luyen cay hoi quy (Gioi han max_depth)
tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X_quad, y_quad)
```

6.3 Trực quan hóa Cây Hồi quy

- Hình bên minh họa cây Hồi quy độ sâu 2.
- Ở mỗi nút lá, giá trị value chính là y trung bình của các mẫu, và mse (Mean Squared Error) là sai số tại nút đó.
- Dự đoán sẽ mang dạng các **bậc thang (step-wise)** vì trong cùng 1 khoảng chia, mọi giá trị x đều trả ra cùng 1 mức y.

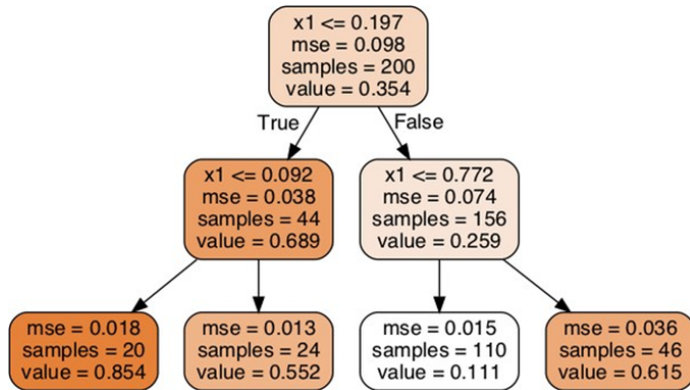


Figure: Hình 6-4: Sơ đồ Decision Tree Regression

6.4 Các Ranh giới Hồi quy (Bậc thang)

- Trái: $\text{max_depth} = 2$ (Có 4 bậc thang).
- Phải: $\text{max_depth} = 3$ (Có 8 bậc thang).
- Cây phân chia không gian thành các mảng và dự đoán y là đường thẳng hằng số trên từng mảng.

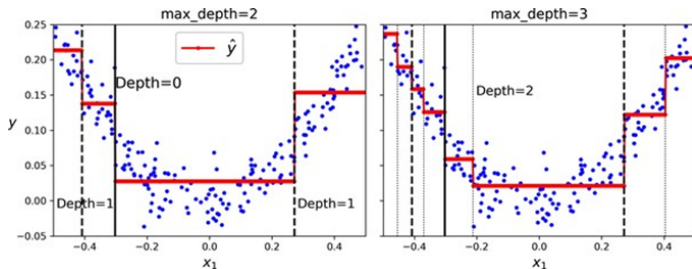


Figure: Hình 6-5: Hàm dự đoán của Cây Hồi quy

6.5 Overfitting trong Hồi quy

- Tương tự như phân loại, Cây Hồi quy rất dễ bị Overfitting nếu không ràng buộc.
- Cây sẽ uốn éo đứt đoạn để đi qua toàn bộ các nhiễu dữ liệu.
- Khắc phục: Dùng `min_samples_leaf=10` để ép mô hình mượt mà hơn.

6.6 Hàm Chi phí CART cho Hồi quy

- Thuật toán CART sẽ tối thiểu hóa sai số MSE thay vì Gini Impurity:

$$J(k, t_k) = \frac{m_{left}}{m} \text{MSE}_{left} + \frac{m_{right}}{m} \text{MSE}_{right}$$

- Trong đó:

$$\text{MSE}_{node} = \frac{1}{m_{node}} \sum_{i \in node} (\hat{y}_{node} - y^{(i)})^2$$

- Và $\hat{y}_{node}$ luôn là Trung bình cộng của các mẫu trong lá.

7.1 Trục chia vuông góc (Orthogonal splits)

- Cây quyết định yêu thích sự đơn giản. Nó luôn phân chia tập dữ liệu bằng các đường **song song hoặc vuông góc** với trục tọa độ.
- Nó không thể vẽ đường chéo hay đường cong!
- Điều này dẫn đến sự nhạy cảm cực độ đối với **Hướng Xoay của Dữ liệu (Axis Orientation)**.